

Module 1

iverilog Deep Dive

Learning objectives

- Master iverilog for Verilog/SystemVerilog testbench development

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["iverilog Overview"]

T2["Compilation Process"]

T1 --> T2

T3["Simulation Execution"]

T2 --> T3

Master iverilog for Verilog/SystemVerilog testbench development

Overview

- This module provides comprehensive coverage of Icarus Verilog (iverilog), an open-source Verilog simulator.

Design architecture

1. iverilog compile-and-run architecture

- Sources: RTL under
 `module1/dut/` plus testbench
 under `examples/` or `tests/`
- Compile: `iverilog` elaborates
 modules, produces a VVP bytecode
 executable
- Simulate: `vvp` drives the event
 scheduler; `\$dumpvars` can emit
 VCD
- Artifacts: `simv` (or custom
 name), `\*.vcd`, compile logs in
 example directories

```
Rtl Architecture
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart LR
SRC[RTL + TB .v files] --> IV[iverilog compile]
IV --> BYTE[VVP bytecode simv]
BYTE --> VVP[vvp execute]
subgraph dut1["module1/dut"]
M2[mux_2to1]
```

2. Module 1 DUT catalog

- ``dut/multiplexers/``: ``mux_2to1``,
``mux_4to1`` — select lines,
combinational outputs
- ``dut/counters/``: ``counter_4bit``
— clock, reset, enable, parallel
load
- Interface pattern: Clock + reset
on sequential blocks; data ports
on combinational paths
- Reuse: Same DUTs exercised
across compilation, simulation,
and testbench examples

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

VEC[directed vectors] --> STIM[initial / tasks]

STIM --> DUT[mux or counter]

DUT --> CHK[self-check if expected]

CHK --> RPT[pass/fail summary]

FILE[file I/O vectors] --> STIM

3. Verilog testbench structure (iverilog)

- Top: ``module tb;` instantiates DUT, declares wires/regs for connectivity
- Stimulus: ``initial`` blocks and tasks drive inputs over simulation time
- Observation: ``$display``, ``$monitor``, and VCD for passive debug
- Closure: ``$finish`` after pass/fail counts or explicit timeout

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
  COMP[make compilation] --> SIM[make simulation]
  SIM --> TB[testbench_basics]
  TB --> WAVE[waveforms + GTKWave]
  WAVE --> DBG[debug narrowed stimulus]
  DBG --> CI[module1.sh --check]
```

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

SRC[RTL + TB .v files] --> IV[iverilog compile]

IV --> BYTE[VVP bytecode simv]

BYTE --> VVP[vvp execute]

subgraph dut1["module1/dut"]

M2[mux_2to1]

Module 1: DUT hierarchy and signal flow.

Compilation Makefile — iverilog targets

```
# Basic compilation
basic_compilation: basic_compilation.v
    @echo "=== Basic Compilation ==="
    $(IVERILOG) -o basic_compilation basic_compilation.v
    $(VVP) basic_compilation

# Compilation with include path
include_path: basic_compilation.v
    @echo "=== Compilation with Include Path ==="
    $(IVERILOG) -I$(DUT_DIR) -o include_path basic_compilation.v
    $(VVP) include_path

# Compilation with macro definition
macro_def: basic_compilation.v
    @echo "=== Compilation with Macro Definition ==="
    $(IVERILOG) -DDEBUG -o macro_def basic_compilation.v

# ...
```

Verilog TB — DUT instantiation

```
module mux_4to1_test;

    //
    =====
    =
    // Testbench Signal Declarations
    //
    =====
    =
    // These signals connect to the DUT and are controlled by the
    testbench

    reg [1:0] sel;          // Select signal (2-bit, drives DUT input)
    reg      in0, in1, in2, in3; // Input signals (drive DUT inputs)
    wire      out;          // Output signal (observes DUT output, wire
because it's driven by DUT)
    reg      clk;          // Clock signal (for demonstration, not
needed
```

VVP simulation Makefile

```
vvp_basics: vvp_basics.v
    @echo "=== Compiling vvp_basics ==="
    $(IVERILOG) -o vvp_basics vvp_basics.v
    @echo "=== Running simulation ==="
    $(VVP) vvp_basics
    @echo "=== Running with verbose mode ==="
    $(VVP) -v vvp_basics
```

DUT — mux_4to1 select logic

```
module mux_4to1(
    input  wire [1:0] sel,  // 2-bit select signal
    input  wire        in0, // Input 0
    input  wire        in1, // Input 1
    input  wire        in2, // Input 2
    input  wire        in3, // Input 3
    output reg         out  // Output (must be reg for procedural
assignment)
);

/**
 * Combinational Logic: 4-to-1 Multiplexer
 *
 * This always block implements the multiplexer logic using a case
statement.
 * The always @(*) syntax means this block is sensitive to all
signals used
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module1/examples/<example>/ — run make run from that directory

Directed test execution sequence (1)

- TB compares expected vs actual
- error count on stdout
- \$finish with summary
- Gather RTL from module1/dut/
- iverilog -o sim tb.v dut.v

Directed test execution sequence (2)

- vvp sim
- read \$display self-check
- \$dumpvars VCD

Run compilation example

```
# Basic compilation
basic_compilation: basic_compilation.v
    @echo "=== Basic Compilation ==="
    $(IVERILOG) -o basic_compilation basic_compilation.v
    $(VVP) basic_compilation
```

VVP runtime invocation

```
vvp_basics: vvp_basics.v
    @echo "=== Compiling vvp_basics ==="
    $(IVERILOG) -o vvp_basics vvp_basics.v
    @echo "=== Running simulation ==="
    $(VVP) vvp_basics
    @echo "=== Running with verbose mode ==="
    $(VVP) -v vvp_basics
```

Self-checking stimulus loop

```
//  
    // Test Strategy:  
    // - Exhaustive testing: Test all 4 select combinations  
    // - For each select value, set the corresponding input high,  
others  
low  
    // - Verify that output matches the selected input  
    //  
    // This pattern (stimulus → wait → check) is fundamental to all  
verification:  
    // 1. Apply stimulus (set inputs)  
    // 2. Wait for propagation (combinational logic settles  
immediately,  
but we wait for clarity)  
    // 3. Check result (compare output to expected value)  
    // 4. Report pass/fail  
    //  
    // In UVM: This evolves into:
```

Verification & testing methods

1. Directed functional testing

- Vector tables: Apply known (sel, d0..d3) or (en, load) patterns; compare `y` or `count`
- Corner cases: Reset during operation, enable deasserted, all MUX select values
- Self-check: Inline `if (expected !== actual)` with error count and final summary

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

COMP[make compilation] --> SIM[make simulation]

SIM --> TB[testbench_basics]

TB --> WAVE[waveforms + GTKWave]

WAVE --> DBG[debug narrowed stimulus]

DBG --> CI[module1.sh --check]

2. File-based and batch testing

- Test vectors: Read stimulus from files (``file_io`` examples) for regression-style reruns
- Golden logs: Compare simulator output to expected transcripts where provided
- Orchestration:
``./scripts/module1.sh`` runs example matrix; ``--check`` for CI-style gate

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

COMP[make compilation] --> SIM[make simulation]

SIM --> TB[testbench_basics]

TB --> WAVE[waveforms + GTKWave]

WAVE --> DBG[debug narrowed stimulus]

DBG --> CI[module1.sh --check]

3. Waveform and debug methodology

- VCD generation: Enable
`\$dumpfile` / `\$dumpvars` in TB
or via plusargs where shown
- GTKWave: Align clock edges,
verify reset release, step
through MUX transitions
- Debug flow: Re-run single
example with `make clean &&
make` and narrowed stimulus

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
  COMP[make compilation] --> SIM[make simulation]
  SIM --> TB[testbench_basics]
  TB --> WAVE[waveforms + GTKWave]
  WAVE --> DBG[debug narrowed stimulus]
  DBG --> CI[module1.sh --check]
```

Self-check — expected vs actual

```
// Print test table header
    $display("\nTesting all select combinations:");
    $display("sel | in0 | in1 | in2 | in3 | out | Expected");
    $display("----|-----|-----|-----|-----|-----|-----");

    //
=====

    // Test Case 1: Select in0 (sel=00)
    //
=====

    // Strategy: Set in0=1, all others=0, sel=00, verify out=in0
    sel = 2'b00;
    in0 = 1'b1; // Set input 0 high
    in1 = 1'b0;
    in2 = 1'b0;
    in3 = 1'b0;
    #5; // Wait for combinational logic to propagate (though it's
```

Syllabus topics

1. iverilog Overview (1/5)

- What is iverilog?
- Open-source Verilog/SystemVerilog simulator
- Part of the Icarus Verilog project
- Cross-platform (Linux, macOS, Windows)
- Free and open-source
- iverilog Capabilities and Features

1. iverilog Overview (2/5)

- Verilog-2001 support
- SystemVerilog support (limited)
- VCD waveform generation
- PLI/VPI support
- Fast compilation and simulation
- Supported Verilog/SystemVerilog Constructs

1. iverilog Overview (3/5)

- Most Verilog-2001 features
- Basic SystemVerilog features
- Interfaces (limited)
- Classes (limited)
- Assertions (limited)
- Limitations and Workarounds

1. iverilog Overview (4/5)

- Limited SystemVerilog support
- Some advanced features not supported
- Performance considerations for large designs
- Workarounds for unsupported features
- When to Use iverilog
- Learning Verilog testbench concepts

1. iverilog Overview (5/5)

- Quick prototyping
- Simple to medium complexity designs
- When you prefer Verilog syntax
- Open-source projects

2. Compilation Process (1/6)

- ****Compilation Command (`iverilog`)**
- Basic syntax: ``iverilog -o output input.v``
- Output file specification
- Input file handling
- Command-Line Options and Flags
- ``-o <file>``: Specify output file

2. Compilation Process (2/6)

- ``-I<directory>``: Add include path
- ``-D<macro>=<value>``: Define macro
- ``-g<spec>``: Specify language generation
- ``-s <top>``: Specify top-level module
- ``-t <target>``: Specify target format
- Include Path Management (``-I``)

2. Compilation Process (3/6)

- Adding include directories
- Multiple include paths
- Relative vs. absolute paths
- Best practices
- Define Macros (``-D``)
- Conditional compilation

2. Compilation Process (4/6)

- Macro definitions
- Default values
- Usage in testbenches
- Timescale Handling
- `timescale directive
- Default timescale

2. Compilation Process (5/6)

- Timescale conflicts
- Resolution
- Multi-File Compilation
- Compiling multiple files
- File ordering
- Dependency management

2. Compilation Process (6/6)

- Library files
- Library Management
- Library files
- Library paths
- Precompiled libraries
- ``basic_compilation.v``: Demonstrates basic compilation, include paths, macros, multi-file compilation

3. Simulation Execution (1/4)

- VVP (Icarus Verilog Runtime) Usage
- What is VVP?
- Running simulations: ``vvp executable``
- VVP vs. other simulators
- Simulation Command and Options
- Basic execution: ``vvp executable``

3. Simulation Execution (2/4)

- Command-line options
- Runtime flags
- Runtime Flags
- ``-v``: Verbose mode
- ``-n``: No execution (syntax check)
- ``-M <path>``: Module path

3. Simulation Execution (3/4)

- ``-m <module>``: Load module
- Simulation Control
- Starting simulation
- Stopping simulation
- Simulation time
- Event scheduling

3. Simulation Execution (4/4)

- Performance Considerations
- Simulation speed
- Memory usage
- Large design handling
- Optimization tips
- ``vvp_basics.v``: Demonstrates VVP simulation execution and options

4. Verilog Testbench Writing for iverilog (1/8)

- Verilog Testbench Structure
- Module-based testbenches (foundation for all testbenches)
- Top-level testbench module (analogous to UVM test class)
- DUT instantiation (part of UVM environment)
- Signal declarations (evolve into UVM interfaces and virtual interfaces)
- Module-Based Testbenches

4. Verilog Testbench Writing for iverilog (2/8)

- Testbench as a module (no ports needed, self-contained)
- Internal signal declarations (signals connect DUT to testbench)
- UVM Connection: In UVM, this structure evolves into environments with agents, drivers, monitors, an...
- Initial Blocks for Test Sequences
- Test sequence organization (foundation for UVM sequences)
- Sequential test application (stimulus generation)

4. Verilog Testbench Writing for iverilog (3/8)

- Timing control (delay and synchronization)
- Multiple initial blocks (concurrent test processes)
- UVM Connection: Initial blocks with test sequences
 evolve into UVM sequences that generate transact...
- Always Blocks for Clock Generation
- Clock generation patterns (essential for sequential logic)
- Continuous clock (runs forever)

4. Verilog Testbench Writing for iverilog (4/8)

- Gated clock (conditional clock)
- Multiple clocks (multi-clock domain designs)
- UVM Connection: Clock generation in UVM is handled by clocking blocks and virtual interfaces
- Signal Access and Monitoring
- Reading signals (observing DUT outputs)
- Writing signals (driving DUT inputs)

4. Verilog Testbench Writing for iverilog (5/8)

- Signal monitoring (continuous observation)
- Signal tracing (debugging)
- UVM Connection: Signal access evolves into UVM monitors that observe transactions and send them to...
- `$display`, `$monitor`, `$strobe`
- ``$display``: Immediate output (prints when called)
- ``$monitor``: Continuous monitoring (prints on signal changes)

4. Verilog Testbench Writing for iverilog (6/8)

- ``$strobe``: End-of-time-step output (prints after all assignments)
- Formatting options (similar to `printf` in C)
- When to use each (timing differences matter)
- UVM Connection: These system tasks are used in UVM for logging and debugging, but UVM also provides...
- File I/O (`$readmemh`, `$readmemb`, `$fopen`, `$fwrite`)
- Reading memory files: ``$readmemh`` (hex), ``$readmemb`` (binary)

4. Verilog Testbench Writing for iverilog (7/8)

- Opening files: ``$fopen`` (returns file handle)
- Writing files: ``$fwrite``, ``$fdisplay`` (formatted output)
- Closing files: ``$fclose`` (cleanup)
- File-based testbenches (separate test data from code)
- UVM Connection: File-based test vectors evolve into transaction-based verification where sequences...
- ``module1/examples/testbench_basics/``:

4. Verilog Testbench Writing for iverilog (8/8)

- ``mux_4to1_test.v``: Demonstrates basic testbench structure, stimulus generation, and checking
- ``counter_test.v``: Demonstrates clock generation, reset sequences, and sequential logic testing
- ``module1/examples/file_io/``: File I/O operations and file-based test patterns

5. Waveform Generation (1/4)

- VCD File Generation (\$dumpfile, \$dumpvars)
- ` \$dumpfile `: Specify VCD filename
- ` \$dumpvars `: Select signals to dump
- Dump levels (0, 1, 2, etc.)
- Signal selection
- Signal Selection for Waveforms

5. Waveform Generation (2/4)

- Dumping all signals
- Dumping specific signals
- Dumping hierarchy levels
- Selective dumping
- GTKWave Compatibility
- VCD file format

5. Waveform Generation (3/4)

- GTKWave viewing
- Signal organization
- Save files (.gtkw)
- Waveform Analysis
- Viewing waveforms
- Signal timing

5. Waveform Generation (4/4)

- Timing relationships
- Debugging with waveforms
- ``waveform_example.v``: Comprehensive waveform generation with different dump levels

6. Debugging with iverilog (1/5)

- Compilation Error Debugging
- Understanding error messages
- Common compilation errors
- Syntax errors
- Missing files
- Include path issues

6. Debugging with iverilog (2/5)

- Runtime Error Debugging
- Understanding runtime errors
- Simulation errors
- Signal value errors
- Timing errors
- Signal Tracing Techniques

6. Debugging with iverilog (3/5)

- Using \$display for tracing
- Using \$monitor for continuous tracing
- VCD file analysis
- Signal value inspection
- Logging Strategies
- Debug levels

6. Debugging with iverilog (4/5)

- Conditional logging
- File-based logging
- Log organization
- Common Pitfalls and Solutions
- Timescale issues
- Signal initialization

6. Debugging with iverilog (5/5)

- Race conditions
- Blocking vs. non-blocking
- Common mistakes
- ``debug_example.v``: Debugging techniques, logging strategies, error handling

7. Advanced iverilog Features (1/5)

- PLI/VPI Basics
- What is PLI/VPI?
- When to use PLI/VPI
- Basic PLI/VPI usage
- Limitations
- System Tasks and Functions

7. Advanced iverilog Features (2/5)

- Built-in system tasks
- Custom system tasks
- Task vs. function
- Usage examples
- User-Defined System Tasks
- Creating custom tasks

7. Advanced iverilog Features (3/5)

- Task registration
- Task implementation
- Integration
- Performance Optimization
- Compilation optimization
- Simulation optimization

7. Advanced iverilog Features (4/5)

- Memory optimization
- Speed optimization
- Large Design Handling
- Compiling large designs
- Memory management
- Incremental compilation

7. Advanced iverilog Features (5/5)

- Design partitioning

8. Project Organization (1/11)

- Makefile Integration
- Basic Makefile structure
- Compilation rules
- Test execution
- Clean targets
- Dependency management

8. Project Organization (2/11)

- Scripting for Automation
- Shell scripts for automation
- Batch processing
- Test automation
- Result collection
- Batch Simulation

8. Project Organization (3/11)

- Running multiple tests
- Test selection
- Parallel execution
- Result aggregation
- Regression Testing Setup
- Test suite organization

8. Project Organization (4/11)

- Regression test structure
- Pass/fail reporting
- Test result tracking
- Location:
 - ``module0/examples/iverilog_basics/and_gate_test.v``
- Demonstrates: Basic testbench structure, signal driving, output monitoring
- Key Concepts: DUT instantiation, stimulus application, response checking

8. Project Organization (5/11)

- UVM Connection: Foundation for UVM environment and driver/monitor patterns
- Location:
 ``module1/examples/testbench_basics/mux_4to1_test.v``
- Demonstrates:
- Module-based testbench structure
- System tasks: ``$display``, ``$monitor``, ``$strobe``
 (timing differences)
- Self-checking testbench (automatic pass/fail detection)

8. Project Organization (6/11)

- Combinational logic testing patterns
- Key Concepts: Stimulus generation, response checking, signal monitoring
- UVM Connection: Patterns evolve into UVM sequences (stimulus) and scoreboards (checking)
- Location:
 ``module1/examples/testbench_basics/counter_test.v``
- Demonstrates:
- Clock generation with ``always`` blocks (essential for sequential logic)

8. Project Organization (7/11)

- Reset sequences (synchronous reset patterns)
- Enable signal control (testing control signals)
- Sequential logic testing (state machine verification)
- Key Concepts: Clock generation, reset sequences, timing verification
- UVM Connection: Clock generation handled by UVM clocking blocks; reset sequences become UVM sequenc...
- Location:
 `module1/examples/file\_io/file\_read\_test.v`

8. Project Organization (8/11)

- Demonstrates:
- Reading test vectors from files (``$readmemh``, ``$readmemb``)
- Writing results to files (``$fopen``, ``$fwrite``, ``$fclose``)
- File-based testbench patterns (scalable testing)
- Key Concepts: Separating test data from code, automated test vector application
- UVM Connection: File-based testing evolves into transaction-based verification where sequences gene...

8. Project Organization (9/11)

- Location: See compilation examples
`(`module1/examples/compilation/`)`
- Demonstrates: Multi-file compilation, include paths, library management
- Key Concepts: Project organization, dependency management
- UVM Connection: Multi-file organization is essential for UVM's component-based architecture
- Location:
``module1/tests/basic_tests/test_mux_4to1.v``
- Demonstrates:

8. Project Organization (10/11)

- Reusable test tasks (modular test organization)
- Comprehensive test coverage (multiple test patterns)
- Automatic pass/fail tracking (test statistics)
- Error reporting (detailed failure messages)
- Test result summary (overall test status)
- Key Concepts: Task-based organization, test statistics, exit codes

8. Project Organization (11/11)

- UVM Connection: Tasks evolve into UVM sequences;
test statistics become UVM test reporting

Hands-on examples

Module 1 self-check

```
$ ./scripts/module1.sh --help
```

Terminal: module1_check

Usage: ./scripts/module1.sh [OPTIONS]

Module 1: iverilog Deep Dive

This script runs examples and tests for Module 1.

OPTIONS:

Examples:

--compilation	Run compilation examples
--simulation	Run VVP simulation examples
--testbench-basics	Run testbench basics examples
--file-io	Run file I/O examples
--waveforms	Run waveform generation examples
--debugging	Run debugging examples
--advanced	Run advanced iverilog features
--all-examples	Run all examples (default)
--skip-examples	Skip all examples

Tests:

--basic-tests	Run basic testbenches
--file-io-tests	Run file I/O testbenches
--all-tests	Run all tests (default)
--skip-tests	Skip all tests

Environment:

--jobs N	Number of parallel build jobs (default: 8)
--no-clean	Skip cleaning before build (faster rebuilds)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all examples and tests
./scripts/module1.sh

# Run only compilation examples
./scripts/module1.sh --compilation
```

Exercise scaffold

```
./scripts/module1.sh --scaffold
```

Demo: Compilation

```
$ cd module1/examples/compilation && make basic_compilation
```

Terminal: ex01_compilation

```
=== Basic Compilation ===  
iverilog -o basic_compilation basic_compilation.v  
vvp basic_compilation  
=====
```

Basic Compilation Example

```
=====
```

Timescale: basic_compilation.v

Compilation successful!

```
=====
```

basic_compilation.v:48: \$finish called at 10000 (1ps)

Demo: Simulation

```
$ cd module1/examples/simulation && make all
```

Terminal: ex02_simulation

```
=== Compiling vvp_basics ===  
iverilog -o vvp_basics vvp_basics.v  
=== Running simulation ===  
vvp vvp_basics  
=====
```

VVP Simulation Basics

```
=====
```

Simulation started at time 0

Time 10000: counter = 1

Time 20000: counter = 2

Time 30000: counter = 3

Time 40000: counter = 4

Time 50000: counter = 5

```
=====
```

Simulation completed at time 50000

```
=====
```

vvp_basics.v:43: \$finish called at 60000 (1ps)

=== Running with verbose mode ===

vvp -v vvp_basics

Compiling VVP ...

... VVP file version 12.0 (stable)

Compile cleanup...

... 2 functors (net_fun pool=524288 bytes)

- 0 logic
- 0 bufif
- 0 resolv
- 2 signals

... 2 filters (net_fil pool=524288 bytes)

... 26 opcodes (24576 bytes)

... 2 nets

... 2 vvp_nets (1048544 bytes)

... 0 arrays (0 words)

... 0 memories

- 0 logic (0 words)
- 0 real (0 words)

... 1 scopes

Demo: Testbench basics

```
$ cd module1/examples/testbench_basics && make all
```

Terminal: ex03_testbench_basics

```
=== Compiling mux_4to1_test ===
iverilog -o mux_4to1_test mux_4to1_test.v ../dut/multiplexers/mux_4to1.v
=== Running simulation ===
vvp mux_4to1_test
=====
4-to-1 Multiplexer Testbench
=====
VCD info: dumpfile mux_4to1_test.vcd opened for output.
Time 0: sel=00, in0=0, in1=0, in2=0, in3=0, out=0

Testing all select combinations:
sel | in0 | in1 | in2 | in3 | out | Expected
----|-----|-----|-----|-----|-----|-----
Time 10000: sel=00, in0=0, in1=0, in2=0, in3=0, out=1
01 | 0 | 1 | 0 | 0 | 1 | in0
Time 15000: sel=01, in0=0, in1=1, in2=0, in3=0, out=1
10 | 0 | 0 | 1 | 0 | 1 | in1
Time 20000: sel=10, in0=0, in1=0, in2=1, in3=0, out=1
11 | 0 | 0 | 0 | 1 | 1 | in2
Time 25000: sel=11, in0=0, in1=0, in2=0, in3=1, out=1

=====
All tests passed!
=====
11 | 0 | 0 | 0 | 1 | 1 | in3
mux_4to1_test.v:208: $finish called at 40000 (1ps)
=== Compiling counter_test ===
iverilog -o counter_test counter_test.v ../dut/counters/counter_4bit.v
=== Running simulation ===
vvp counter_test
=====
4-bit Counter Testbench
=====
VCD info: dumpfile counter_test.vcd opened for output.

Test 1: Reset (rst_n=0)
```

Demo: File I/O

```
$ cd module1/examples/file_io && make all
```

Terminal: ex04_file_io

```
=== Compiling file_read_test ===
iverilog -o file_read_test file_read_test.v ../../module0/dut/simple_gates/and_gate.v
=== Running simulation ===
vvp file_read_test
=====
File I/O Testbench Example
=====
WARNING: file_read_test.v:103: Excess hex digits (1 of '00') while reading 2-bit words.
Read test vectors from file
Test vectors:
  Vector 0: a=0, b=0, expected=0
  Vector 1: a=0, b=1, expected=0
  Vector 2: a=0, b=0, expected=0
  Vector 3: a=0, b=1, expected=1
Time 5000: a=0, b=0, y=0, expected=0
Time 10000: a=0, b=1, y=0, expected=0
Time 15000: a=0, b=0, y=0, expected=0
Time 20000: a=0, b=1, y=0, expected=1
ERROR: file_read_test.v:164: Test failed at vector 3: expected 1, got 0
      Time: 20000 Scope: file_read_test
```

Results written to test_results.txt

View results with: cat test_results.txt

```
=====
```

file_read_test.v:181: \$finish called at 30000 (1ps)

=== Checking output file ===

Results file created:

Test Results

```
=====
```

Time	a	b	y	Expected	Pass/Fail
------	---	---	---	----------	-----------

5000	0	0	0	0	PASS
------	---	---	---	---	------

10000	0	1	0	0	PASS
-------	---	---	---	---	------

15000	0	0	0	0	PASS
-------	---	---	---	---	------

20000	0	1	0	1	FAIL
-------	---	---	---	---	------

Demo: Waveforms

```
$ cd module1/examples/waveforms && make all
```

Terminal: ex05_waveforms

```
=== Compiling waveform_example ===
iverilog -o waveform_example waveform_example.v ../dut/multiplexers/mux_4to1.v
=== Running simulation ===
vvp waveform_example
=====
Waveform Generation Example
=====
VCD info: dumpfile waveform_example.vcd opened for output.
VCD warning: ignoring signals in previously scanned scope waveform_example.dut.
Time 60000: sel=00, out=1
Time 100000: sel=01, out=1
Time 140000: sel=10, out=1
Time 180000: sel=11, out=1
=====
VCD file generated: waveform_example.vcd
View with: gtkwave waveform_example.vcd
=====
In GTKWave, you can:
- Navigate signals in the hierarchy
- Zoom in/out to see timing details
- Add signals to the waveform view
- Measure timing between signals
=====
waveform_example.v:106: $finish called at 200000 (1ps)
=== VCD file generated ===
VCD file: waveform_example.vcd
-rwxrwxrwx 1 yongfu yongfu 1023 May 31 02:09 waveform_example.vcd
View with: gtkwave waveform_example.vcd
```

Demo: Debugging

```
$ cd module1/examples/debugging && make all
```

Terminal: ex06_debugging

```
=== Compiling debug_example ===
iverilog -o debug_example debug_example.v ../module0/dut/simple_gates/and_gate.v
=== Running simulation ===
vvp debug_example
=====
Debugging Example
=====
[DEBUG L0]                Starting test sequence
[DEBUG L1]                Test case 1: a=0, b=0
VCD info: dumpfile debug_example.vcd opened for output.
[DEBUG L1]                Test case 2: a=1, b=1
[DEBUG L0]                Test sequence completed
=====
Debugging demonstration complete
Try changing debug_level to see different verbosity
=====
debug_example.v:78: $finish called at 20000 (1ps)
```

Practice & assessment

What you should know (1/8)

- Compile Verilog designs with iverilog
- Execute simulations with vvp
- Write Verilog testbenches compatible with iverilog
- Generate and analyze waveforms
- Debug iverilog compilation and simulation issues
- Organize projects using iverilog

What you should know (2/8)

- Automate verification flows with iverilog
- Use compilation examples
- Try different compilation options
- Understand compilation process
- Complete testbench for 4-to-1 multiplexer
- Test all input combinations

What you should know (3/8)

- Verify correct operation
- Read test vectors from file
- Apply vectors to DUT
- Write results to file
- Compare expected vs. actual
- Generate VCD files

What you should know (4/8)

- View waveforms in GTKWave
- Analyze signal timing
- Debug using waveforms
- Create Makefile for compilation
- Add test execution rules
- Implement clean targets

What you should know (5/8)

- Organize project structure
- Can compile Verilog designs with various iverilog options
- Can execute simulations with vvp
- Can write complete Verilog testbenches
- Can use file I/O in testbenches
- Can generate and view waveforms

What you should know (6/8)

- Can debug compilation and simulation errors
- Can organize projects with Makefiles
- Can automate verification flows
- Prerequisites: [Module 0: Installation and Setup](MODULE0.md)
- Next Steps: [Module 2: Verilator Deep Dive](MODULE2.md) - Master Verilator for C++ testbench develo...
- Next Steps: [Module 3: Testbench Fundamentals](MODULE3.md) - Learn fundamental testbench concepts f...

What you should know (7/8)

- UVM Connection: [UVM Core Repository](<https://github.com/universal-verification-methodology/core>)
- Module 2: Verilator Deep Dive - Master Verilator for C++ testbench development
- Module 3: Testbench Fundamentals (Verilog and C++) - Learn fundamental testbench concepts for both...
- Icarus Verilog Documentation: <http://iverilog.wikia.com/>
- GTKWave Documentation: <http://gtkwave.sourceforge.net/>
- IEEE 1364-2005 Standard: Verilog Hardware Description Language

What you should know (8/8)

- IEEE 1800-2017 Standard: SystemVerilog Language Reference Manual

Assessment checklist

- Can compile Verilog designs with various iverilog options
- Can execute simulations with vvp
- Can write complete Verilog testbenches
- Can use file I/O in testbenches
- Can generate and view waveforms
- Can debug compilation and simulation errors

Summary & next steps

- Master iverilog for Verilog/SystemVerilog testbench development
- Complete module1/CHECKLIST.md
- Review module1/EXAMPLES.md and run each lab
- Next: Next module in course